Exp5: Use Docker Compose to set up a multi-container application with a web server and a database.

Aim

To use Docker Compose to deploy a multi-container application consisting of a **web server** and a **database**.

Software Requirements

- Ubuntu 22.04 LTS
- Docker Engine
- Docker Compose plugin
- Web browser

Application Overview

Architecture:

Browser → Web Container (Flask)

|

↓

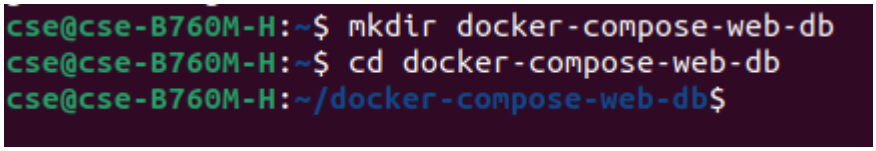
Database Container (MySQL)

- *Two containers*
- *One network (created automatically)*
- *Communication using* **service names**

Step 1: Create Project Directory

mkdir docker-compose-web-db

cd docker-compose-web-db



Step 2: Create Web Application Code (Python Flask)

Create **app.py**:

```
nano app.py
```

Paste:

```
from flask import Flask
import mysql.connector

app = Flask(__name__)

@app.route("/")
def hello():
    db = mysql.connector.connect(
        host="db",
        user="root",
        password="root",
        database="testdb"
    )
    cursor = db.cursor()
    cursor.execute("SELECT 'Hello from MySQL inside Docker Compose!'")
    result = cursor.fetchone()
    return result[0]

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

```
  GNU nano 6.2
from flask import Flask
import mysql.connector

app = Flask(__name__)

@app.route("/")
def hello():
    db = mysql.connector.connect(
        host="db",
        user="root",
        password="root",
        database="testdb"
    )
    cursor = db.cursor()
    cursor.execute("SELECT 'Hello from MySQL inside Docker Compose!'")
    result = cursor.fetchone()
    return result[0]

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Save and exit.

Step 3: Create Requirements File

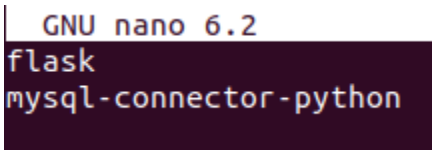
Create **requiremets.txt:**

```
nano requirements.txt
```

Paste:

```
flask

mysql-connector-python
```



Save and exit.

Step 4: Create Dockerfile for Web Service

Create **Dockerfile**:

```
nano Dockerfile
```

Paste:

```
FROM python:3.10-slim


WORKDIR /app


COPY requirements.txt .

RUN pip install -r requirements.txt


COPY app.py .


EXPOSE 5000


CMD ["python", "app.py"]
```

```
  GNU nano 6.2
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Save and exit.

Step 5: Create docker-compose.yml

```
nano docker-compose.yml
```

Paste:

```
version: "3.8"


services:

 web:

    build: .

    ports:

      - "5000:5000"

    depends_on:

      - db


 db:

    image: mysql:8.0

    environment:

      MYSQL_ROOT_PASSWORD: root

      MYSQL_DATABASE: testdb
```

```
  GNU nano 6.2
version: "3.8"

services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - db

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: testdb
```

Save and exit.

Step 6: Start the Multi-Container Application

```
docker-compose up
```

It takes some time. So let us wait until we see:

```
Ready for connections
```

OR

```
Running on http://0.0.0.0:5000
```

```
db_1   | 2026-02-20T06:14:17.101917Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::' port: 33060, socket: /var/run/mysqld/mysqlx.sock
db_1   | 2026-02-20T06:14:17.101943Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.45'  socket: '/var/run/mysqld/mysqld.sock'  port: 3306  MySQL Community Server - GPL.
web_1  | 172.19.0.1 - - [20/Feb/2026 06:47:47] "GET / HTTP/1.1" 200 -
web_1  | 172.19.0.1 - - [20/Feb/2026 06:47:48] "GET / HTTP/1.1" 200 -
```

Note: *The above command 'up':*

- *Runs in foreground mode*
- *Keeps showing live logs*
- *Does not return control to the prompt*

Step 7: Verify Running Containers

Open another Terminal window by:

- Ctrl + Alt + T
  OR

- Right-click → *Open Terminal*

Go to the same project directory:

```
cd docker-compose-web-db
```

```
docker-compose ps
```

```
cse@cse-B760M-H:~$ cd docker-compose-web-db/
cse@cse-B760M-H:~/docker-compose-web-db$ docker-compose ps
         Name                    Command           State           Ports
-------------------------------------------------------------------------------
docker-compose-web-      docker-entrypoint.sh     Up      3306/tcp, 33060/tcp
db_db_1                  mysqld
docker-compose-web-      python app.py            Up      0.0.0.0:5000-
db_web_1                                                  >5000/tcp,:::5000-
                                                          >5000/tcp
cse@cse-B760M-H:~/docker-compose-web-db$
```

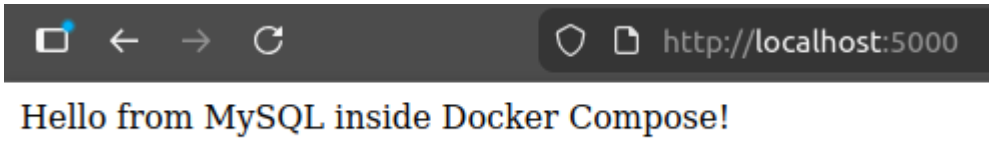
Step 8: Access the Web Application

Open browser and visit:

```
http://localhost:5000
```

Expected Output:

```
Hello from MySQL inside Docker Compose!
```

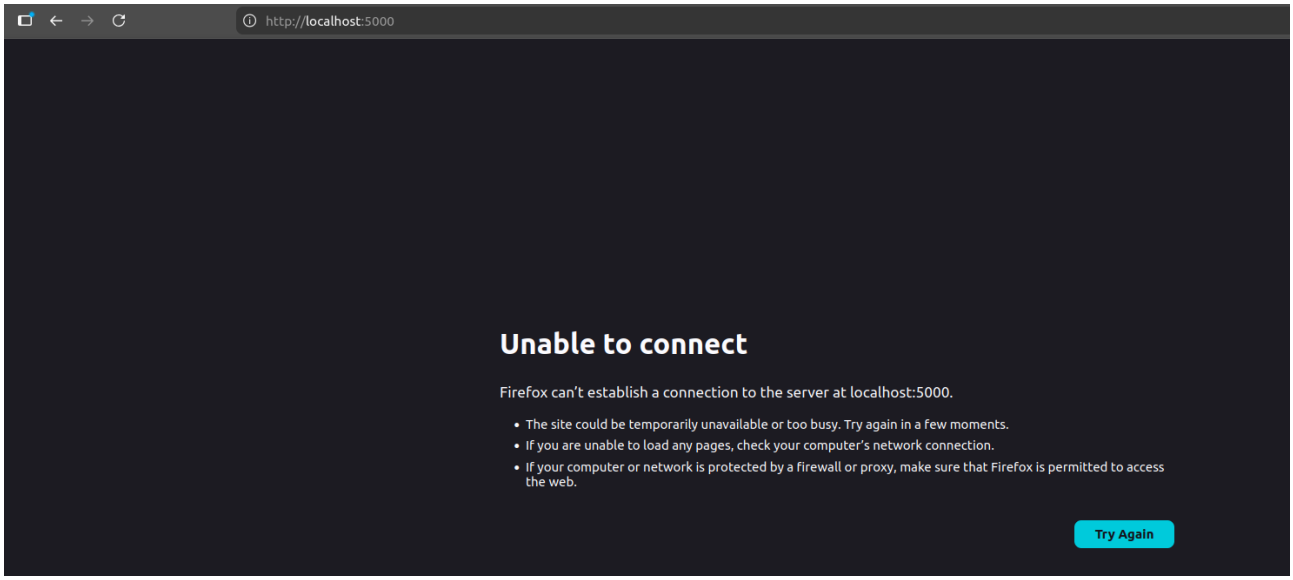


Step 9: Stop and Remove the Containers

Go to 2nd terminal and run:

```
docker-compose down
```

```
cse@cse-B760M-H:~/docker-compose-web-db$ docker-compose down
Stopping docker-compose-web-db_web_1 ... done
Stopping docker-compose-web-db_db_1  ... done
Removing docker-compose-web-db_web_1 ... done
Removing docker-compose-web-db_db_1  ... done
Removing network docker-compose-web-db_default
cse@cse-B760M-H:~/docker-compose-web-db$
```

```
db_1    | 2026-02-20T06:14:17.101943Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.45'  socket: '/var/run/mysqld/my
web_1   | 172.19.0.1 - - [20/Feb/2026 06:47:47] "GET / HTTP/1.1" 200 -
web_1   | 172.19.0.1 - - [20/Feb/2026 06:47:48] "GET / HTTP/1.1" 200 -
docker-compose-web-db_web_1 exited with code 137
db_1    | 2026-02-20T06:52:25.769657Z 0 [System] [MY-013172] [Server] Received SHUTDOWN from user <via user signal>. Shutting down mysqld (Version: 8.0.45).
db_1    | 2026-02-20T06:52:27.040328Z 0 [System] [MY-010910] [Server] /usr/sbin/mysqld: Shutdown complete (mysqld 8.0.45)  MySQL Community Server - GPL.
docker-compose-web-db_db_1 exited with code 0
cse@cse-B760M-H:~/docker-compose-web-db$
```

Now go back to browser and refresh the page to confirm that the application is not running.